INTRO

Hi, I'm Swapnil Pathak, a Full Stack Java Developer at TCS with around 4.6 years of experience.
I have worked on public-sector projects like Passport Seva and ePashudhan.
In Passport Seva, I worked on the User Management module for user creation and role-based access.
My responsibilities included REST APIs, business logic, database interaction, validations, exception handling, and React UI integration.
In my current ePashudhan project, I mainly work on backend development using Spring Boot and microservices.
My responsibilities include REST APIs, business logic, database interaction, Redis cache management, and CAPTCHA implementation.
My key technical skills are Java, Spring Boot, JPA, Hibernate, Spring Security, Microservices, React.js, Axios, and Redux.
For service-to-service communication, I have worked with REST APIs and RestTemplate.
I am now looking for opportunities to work on large-scale applications, strengthen my microservices skills, take more ownership, and continue growing technically.

Project

Passport Seva is a government platform that provides various passport-related services to citizens, such as passport application and processing.
In an e-Passport, an embedded electronic chip is used to securely store the passport holder's information and biometric details, which can be electronically verified.
ePashudhan is a digital platform focused on livestock management and veterinary healthcare services.
It provides functionalities such as owner registration, animal registration, veterinary services, and other livestock-related activities.

Explain your architecture

We followed a microservices-based architecture in Passport Seva.
The frontend was developed using React.js and communicated with backend services through REST APIs.
The backend had multiple Spring Boot microservices, with each service handling different business functionalities.
Service-to-service communication was mainly done through REST APIs using HTTP calls.
We used MySQL and DB2 for database and data persistence.
Redis was used for caching, and Elasticsearch was used for search functionality.
The application was deployed on Red Hat OpenShift, which is based on Kubernetes.
My main responsibility was developing REST APIs, implementing business logic, database interaction, validations, exception handling, and React API integration.

Explain your deployment process

My main responsibility was application development; deployment was handled by the DevOps and architecture teams.
After development and local testing, I pushed the code to Git and created a Pull Request for code review.
Once the code was approved and testing was completed, the DevOps team handled the CI/CD build and deployment process.
The DevOps team triggered the Jenkins build and generated the required build artifacts.
These artifacts were then handed over to the architecture team for the deployment process.
The application was deployed on the Red Hat OpenShift platform.
I did not personally trigger or configure the Jenkins pipelines or OpenShift deployment.
After deployment, I verified the application functionality and supported troubleshooting if any issues occurred.
For production monitoring and troubleshooting, I used AppDynamics to check application errors and help identify the root cause.

Red Hat OpenShift and Kubernetes.
OpenShift is an enterprise platform from Red Hat built on top of Kubernetes.
It is used to deploy, manage, and scale containerized applications.
Kubernetes is a container orchestration platform used to manage containerized applications.
It manages things like Pods, deployments, services, and replicas.
The deployment configuration defines the application image and the desired number of replicas.
OpenShift then creates pods to match that desired state.
For example, if an application needs 3 instances, Kubernetes can maintain 3 running Pods.
If one Pod goes down, Kubernetes can create a replacement Pod to maintain the required number of replicas.

What is a Pod?
A Pod is the smallest deployable unit in Kubernetes and OpenShift.
A Pod contains one or more containers that run together.
In a microservices application, a Spring Boot service can run inside a container, and that container runs inside a pod.
Pods can also communicate with other components through Kubernetes or OpenShift networking.

How did you scale your microservices application?
We scaled our microservices application horizontally by running multiple instances of a service.
The application was deployed on Red Hat OpenShift, where each microservice could have multiple Pods (replicas).
When traffic increased, OpenShift/Kubernetes could increase the number of Pod replicas for that service.
A Service and Load Balancer distributed incoming requests across the available Pods.
Since our microservices were stateless, any Pod could handle any incoming request.
We also used Redis for caching frequently accessed data, which reduced database load and improved performance during high traffic.

Suppose the Owner Registration service receives a large number of requests.
Initially, the service is running with 2 Pods.
During peak traffic, OpenShift scales it to 5 Pods.
Incoming requests are automatically distributed across all 5 Pods.
This improves response time and allows the application to handle more users without changing the code.

How did you identify that a bug is present in which microservice?
First, I check the API Gateway logs to identify the request, HTTP status code, timestamp, and the service to which the request was routed.
Then, I check the centralized logs of the involved microservice and look for exceptions, error messages, and stack traces.
If the request involves multiple microservices and the flow is complex, I use distributed tracing to track the request across different services and identify where the failure occurred.
Once I identify the problematic service, I debug its business logic, database interaction, or service-to-service communication to find the root cause.

How do you know which microservice is down?
First, I check the API Gateway or service logs to see whether requests to a particular microservice are failing, timing out, or returning 5xx errors.
Then, I check the centralized logs and health status of that specific microservice to confirm whether it is running or unavailable.
I use AppDynamics to check the service health, error rate, response time, and availability, which helps identify the affected service.
If the application is running on OpenShift, the DevOps team can also check the Pod status to see whether Pods are running, restarting, or failing.

How do you design your microservice?
I design a microservice around a specific business capability, keeping it loosely coupled and independently deployable.
Service decomposition: I identify the business domains and create separate services such as User, Owner, Animal, or Order Service. Each service owns its business logic and preferably its own data.
Communication: For synchronous operations where an immediate response is required, I use REST APIs; for asynchronous or event-driven operations, I use Kafka.
Common infrastructure: I can use an API Gateway for routing, Service Discovery for locating services, and centralized configuration for managing application configurations.
Reliability and scalability: I keep services stateless where possible and use load balancing, Redis caching, timeouts, retries, and circuit breaker mechanisms as required.
Monitoring and security: I use centralized logging and monitoring tools such as AppDynamics and implement authentication and authorization using Spring Security.

Production issue you faced?
One production issue was related to Aadhaar validation for migrated users.

The Aadhaar validation API was expected to return only one login ID for an Aadhaar number.
In one case, the database query returned two login IDs, which caused a 500 Internal Server Error because the backend expected a single String result.
I investigated both users and found that one login ID was inactive, but its Aadhaar UUID was still mapped.
I identified this incorrect data mapping as the root cause.
After following the required production process, I set the Aadhaar UUID to null for the inactive user.
I verified the data and confirmed that the API was returning the expected single login ID.
The main learning was to check both application logic and production data while troubleshooting issues.

JWT AUTHENTICATION FLOW

When a user logs in, React sends the username and password to the authentication API.
Spring Security validates the credentials using AuthenticationManager and UserDetailsService.
If valid, the backend generates an access JWT and a refresh JWT.
JWT has 3 parts: header, payload, and signature.
The access token is valid for about 30 minutes, while the refresh token is valid for 24 hours.
React stores both tokens in sessionStorage.
For every API request, the access token is sent in the Authorization header as a Bearer token.
Spring Security filter extracts and validates the access token; the signature and expiry are checked.
If valid, the authentication object is stored in the SecurityContext, and the request proceeds to the controller.
If the access token expires, React sends the refresh token to the refresh-token API.
The backend validates the refresh JWT; the signature and expiry are checked.
If valid, the backend generates a new access token and sends it to React.
React updates the access token in sessionStorage and uses it for subsequent requests.
If the refresh token is invalid or expired, the user needs to log in again.
JWT authentication is stateless; the server does not maintain a session for each user because every request contains information needed to authenticate the user.

How did you manage role-based access?

In the User Management module, we managed users and their roles based on their responsibilities in the application.
When creating or updating a user, the Admin could assign a role, and the role information was stored in the database.
During login, the user's role information was loaded and included in the authentication details/JWT claims.
For secured APIs, we used Spring Security for role-based authorization.
We used security rules and annotations like @PreAuthorize to specify which roles could access a particular API.
For example, an Admin API could be configured with @PreAuthorize("hasRole('ADMIN')"), so only users with the Admin role could access it.

Spring Security checks the authenticated user's role before allowing access to the API.
If the user has the required role, the request is allowed; otherwise, Spring Security rejects the request with an authorization error.

How did you handle exceptions?

We handled exceptions centrally using @RestControllerAdvice, instead of writing try-catch blocks in every controller.
We extended ResponseEntityExceptionHandler to handle common Spring exceptions.
For example, for @Valid request-body validation, Spring throws MethodArgumentNotValidException.
We handled it globally by overriding handleMethodArgumentNotValid() and extracted the validation message.
For business-specific errors, we created custom exceptions and handled them using @ExceptionHandler.
We returned a common ApiResponseCommon structure containing details like a success flag, message code, description, timestamp, and error details.
This gave all our APIs a consistent error response and kept the controller and service code clean.
The main benefit of global exception handling is less duplicate code, consistent responses, and easier maintenance.

How did you optimize your React application?

We mainly focused on reducing unnecessary re-renders and unnecessary API calls.
We divided the application into reusable components and managed the required state using Redux.
For large components, we used lazy loading and code splitting so that everything was not loaded at once.
When APIs returned large amounts of data, we used pagination instead of loading all the data together.
We also avoided keeping unnecessary state at the parent level and used React.memo, useMemo, and useCallback wherever they were actually required.

Did you integrate AI in your project?7

Yes, I have worked on an AI-based chatbot using Spring AI and RAG.
In e-Pashudhan, we have a Knowledge Management section that contains documents about different modules and their functionalities.
For example, it has information about user creation and role assignment in Admin, and animal and owner registration in Animal Management.
We used these documents as the knowledge source for the chatbot.
We converted the document content into embeddings and stored them in a vector database.
Basically, a vector database helps us find information based on the meaning of the query.
When a user asks a question, we convert the question into an embedding and search the vector database for the most relevant information.

This is where RAG comes in. It retrieves the relevant information and provides it as context to the LLM.
The LLM, or Large Language Model, uses the user's question and this context to generate the final answer.
Finally, the Spring Boot API sends the response back to the React frontend.

REDIS
We mainly used Redis for master data APIs like Get States and Get Districts.
These APIs were called frequently, but the data changed very rarely, so caching was useful.
We used Spring Cache with @Cacheable.
On the first request, data comes from the database and is stored in Redis.
For the next requests, data is directly returned from Redis, so the repository is not called every time.
This reduces database load and improves the API response time.
When the master data changes, we use @CacheEvict to invalidate the stale data.
If Redis goes down, the application can fetch data directly from the database, so the API can still work.
The main impact is higher response time and more database load because every request may hit the database.
We handle this by monitoring Redis health and configuring proper timeouts so the application can gracefully fall back to the database.

Redis Epashudhan Example
In my case, I worked on a village-related API where we stored village details such as range, latitude, and longitude in the cache.
I had to create an Update API to update these details in the database.
After the update, the old Redis data became stale, so I needed to remove that specific cache entry.
The cache key was dynamic because it was built using values like village code, tehsil code, and other details available at runtime.
Because the key was dynamic, I used Spring CacheManager to access the cache and evict the generated key after the database update.
I could not simply use @CacheEvict with a fixed key because the actual key was constructed dynamically at runtime.
This ensured that the next request would fetch the latest data from the database and store the updated value in Redis.

RestTemplate Communication
In our microservices, we used RestTemplate when one service needed to communicate with another service.
Basically, it is a Spring-provided client that helps us make HTTP calls to another REST API.
For example, if the Owner Service needs some information from the Animal Service, the Owner Service can use RestTemplate to call the Animal Service and get the response.
We can use it for common operations like GET, POST, PUT, and DELETE.

If the other service is down or not responding, the call can fail or take too long and result in a timeout.
So, we handle these cases using proper exception handling and timeout configuration.
For a retry mechanism, we can use Resilience4j to retry the request a few times if the failure is temporary.
For fallback, we can use Resilience4j Circuit Breaker to return a default response or an alternative response when the target service is continuously unavailable.
For example, if the Animal Service is temporarily down, we can retry the call. If it still fails, the fallback can return a proper message instead of making the entire request fail.

Kafka
It is a distributed messaging platform used for asynchronous communication between services.
A Kafka Producer is the service that creates and sends messages or events to Kafka.
The Producer sends the message to a Kafka Broker.
A Kafka Broker is a Kafka server that receives, stores, and serves messages.
The Broker stores the message inside a Topic, which is divided into Partitions.
Partitions allow Kafka to distribute messages and process them in parallel.
Consumers then read messages from these partitions.
Suppose a topic has 3 partitions and the consumer group has 5 consumers.
Only 3 consumers can actively consume because there are only 3 partitions.
The remaining 2 consumers stay idle.
If one active consumer goes down, Kafka rebalances and assigns its partition to another available consumer.
An Offset is the position of a message inside a partition.
Kafka uses the offset to track how far a consumer has processed the partition.
Consumer group offsets are stored in Kafka's internal __consumer_offsets topic.
If a consumer processes a message but crashes before committing the offset, the message can be processed again.
This is called at-least-once processing — the message is processed at least once, but duplicates can happen.

Kafka Epashudhan Example
We mainly used synchronous REST APIs when the client needed an immediate response.
For operations that did not require an immediate response, we used Kafka for asynchronous communication.
For example, when an owner is successfully registered, the Owner Service saves the owner details in the database and immediately returns a success response.
At the same time, it publishes an `OwnerRegistered` event to a Kafka topic.
Different services can consume this event independently — the Notification Service can send an SMS and email, while the Audit Service stores the activity for tracking.
This reduces direct dependency between services because the Owner Service does not need to directly call each service.
If a consumer service is temporarily unavailable, Kafka can retain the event, and the service can process it when it becomes available.

If Kafka is unavailable, the event cannot be published, so depending on the implementation, we can use retry or logging mechanisms to handle the failure.
Kafka was part of our project architecture, but my direct contribution was limited mainly to producer-side integration and understanding event publishing and consumption.
My day-to-day work was mainly around Spring Boot APIs, Spring Security, Redis, and PostgreSQL.

ELASTIC SEARCH

We used Elasticsearch because we needed fast searching across a large amount of passport data.
Elasticsearch is better suited for full-text search, filtering, and relevance-based results.
When data is indexed, Elasticsearch analyzes the text and breaks it into smaller words called tokens.
For example, "Swapnil Pathak" can be tokenized into two tokens: `swapnil` and `pathak`.
These tokens are stored in an inverted index, which helps Elasticsearch quickly find matching records.
When a user searches for "Swapnil Pathak", Elasticsearch also analyzes the query into tokens and searches for those tokens to find the most relevant matching records.
We had around 10 million passport records, so the data was distributed across multiple primary shards.
For example, with 5 primary shards, the search can be performed across those shards in parallel, and Elasticsearch combines the results.
This improves search performance and allows the system to scale by adding more nodes.
For high availability, we use replica shards. If a node containing a primary shard fails, a replica can be promoted to primary.

Transaction Management

It is a concept that ensures all DB operations execute successfully as a single unit
For example, money was debited from bank A and was not credited to bank B due to a database failure; money was lost
Transaction management prevents this by rolling back both operations

@Transactional

It implements the transaction management concept
It commits on success and rolls back the transaction if an exception occurs
We usually use it at the Service layer because that is where our business logic is written.
Internally, Spring uses AOP and creates a proxy around the Service class.
When a @Transactional method is called through the proxy, Spring starts a transaction before executing the method.
If the method completes successfully, the transaction is committed.
If a runtime exception occurs, the transaction is rolled back, so partial changes are not saved.
After that, Spring closes the transaction and returns the response.
One important point is that if a transactional method calls another transactional method within the same class, the call bypasses the Spring proxy.

Because the proxy is bypassed, Spring's transaction management is not applied separately to that internal method call.

Hash-Map

It has key-value pairs where the key is unique, and the value can be duplicated.
The average initial capacity is 16
When we put key-value pairs into a HashMap, Java calculates the hash code of the key using hashCode()
This hash code is used to determine the index of the bucket in an internal array
If the bucket is empty, the entry is stored
If another entry exists at the same index(collision), the new entry is stored in either a linked list / red-black tree
For retrieval, the hash is calculated again, the bucket is identified, and equals() is used to find the same key

Why must we override @equals and @hashCode together?

We should override equals() and hashCode() together because they work together in collections like HashMap and HashSet.
hashCode() is used first to identify the bucket where the object should be stored.
Then equals() is used to compare objects within that bucket and check whether they are logically equal.
If we override only equals() and not hashCode(), two logically equal objects can have different hash codes.
They may then go into different buckets, and HashMap can treat them as different keys.
This violates the Java contract: if two objects are equal according to equals(), they must return the same hashCode().

ACID

Four key properties of DB transactions
Atomicity: it states that all operations in a transaction are considered as a single unit; either all succeed or none
Consistency: It states that data must be valid before and after a transaction and should follow all the constraints.
Isolation: It states that multiple transactions do not interfere with each other; each works independently
Durability: It states that after a transaction, data should remain saved even if the system crashes due to a database failure

SOLID

5 OOP design principles:
Single responsibility principle
It states that a class should have only one reason to change
Open-closed principle
A class should be open for extension but closed for modification

Liskov Substitution Principle
Child classes should be replaceable with parent classes without affecting parent behavior
Interface segregation principle
Clients should not be forced to implement unused interfaces
Dependency inversion principle
High-level modules depend on abstraction, not concrete implementation

Mono vs Micro
In mono, the app is deployed as a single unit; components are tightly coupled, easier to develop initially, but harder to maintain
One failure affects the whole app
In micro, the app is split into independent services, loosely coupled; each service has its own DB and deployment, making it easier to scale, maintain, and deploy independently.
Failure in one service does not affect others

API gateway
It is the entry point for all client requests in a microservices architecture
It routes requests to services based on the request path or headers
It handles authentication, authorization, rate limiting, and logging
For example, Google Cloud Endpoints, azure api management

Circuit breaker pattern
It is a pattern that prevents the system from calling failing services repeatedly
It stops requests when failures exceed a certain threshold
It moves through closed, open, and half-open states
It improves fault tolerance and system stability
It is commonly implemented using Resilience4J or Hystrix
for example
Order service calls the unavailable inventory service repeatedly
It will lead to increased latency, resource exhaustion, and cascading failures
In a circuit breaker, a threshold is set.
If a request fails and exceeds this threshold, the circuit breaker trips to the open state
In this state, requests are rejected with a predefined fallback UI
After a time period, it changes to a half-open state, where limited requests are allowed to see if the inventory service has recovered
If test requests are successful, it changes to a closed state, allowing normal traffic to resume
If not, it changes to the open state again

Saga Pattern
It is a transaction management pattern used in microservices where each service executes its local transaction and publishes an event.
If a step fails, compensating transactions are executed to undo previous changes
types:
Choreography-based: services communicate through events

Orchestration-based: a central orchestrator controls the workflow
For example, the flow goes from the order service to the payment service, but at the inventory service, it fails, and payment will be refunded through a compensating transaction

Load balancing
It is the process of distributing incoming requests(traffic) across multiple instances of a service to avoid overloading a single instance
It increases fault tolerance and prevents server overload
it can be client side(spring cloud load balancer), or server side(nginx)

== vs equals()
== compares the memory address of two objects
equals() compares the actual content/values of the object

hashmap vs concurrent hashmap
HashMap is not thread-safe and is suited for a single-threaded application; it throws a ConcurrentModificationException when accessed by multiple threads
ConcurrentHashMap creates a copy of the HashMap and supports concurrent access by multiple threads, so it is thread-safe

Java 8 features
Lambda expression: enables functional programming and reduces boilerplate code
Stream api: process collections using operations like filter(), map(), collect(), etc
Functional interface: interface with single method(for example, predicate, function)
Method references: a shorter way to refer to a method using the operator (**::**)
Default methods: allow method implementation inside interfaces
Optional class: helps avoid NullPointerException

JPA
It stands for java persistence api
It is a Java specification used for managing relational data in a Java application using objects
It is only a specification; Hibernate is the tool that implements JPA
Hibernate translates your methods into appropriate SQL queries and interacts with the database.
@Entity, @Table, @OneToMany

Volatile keyword
It ensures that changes made to a variable by one thread are immediately visible to all other threads.
Without volatile, threads may cache variables locally, leading to inconsistent data.

Api flow
Suppose the client sends GET /employees/101.
First, the request reaches the embedded Tomcat server.

Before reaching the controller, it passes through the configured filters, including the Spring Security filter chain if security is enabled.
After that, the request reaches DispatcherServlet, which is the front controller of Spring MVC.
DispatcherServlet identifies the correct controller method based on the URL mapping, such as EmployeeController.getEmployee().
Spring extracts 101 from the URL using @PathVariable and passes it to the controller.
The Controller calls the Service layer, where the business logic is handled.
The Service calls the Repository, which uses JPA/Hibernate to fetch the data from the database.
The response comes back in the reverse order: Repository → Service → Controller.
Spring converts the Employee object into JSON, and DispatcherServlet sends the HTTP response back to the client.
DispatcherServlet mainly handles request routing and dispatching.
Authentication and authorization are generally handled earlier by the Spring Security filter chain.

Why are strings immutable?

When a string literal is declared (String str = 'hello'), the Java compiler checks the string pool.
If the string already exists in the pool, a reference to the existing object is assigned to the variable.
If the string is not found in the pool, a new String object is created and added to the pool.
Thus, strings are immutable, and this optimizes memory usage by avoiding creating multiple identical String objects.

Functional interface vs Abstract classes

A functional interface can have only one abstract method and is used with lambda expressions, whereas an abstract class can have multiple abstract and concrete methods
A class can implement multiple interfaces, but a class can extend only one abstract class

Idempotency

An operation is idempotent if performing it multiple times produces the same result as performing it once.
example:
GET: Idempotent (only retrieves data)
PUT: Idempotent (updating the same resource multiple times results in the same state)
DELETE: Idempotent (deleting an already deleted resource does not change the outcome)
POST: Not Idempotent (multiple requests may create multiple resources)

Multi-threading

It is the process of executing multiple threads concurrently within a single process
It enables parallel task execution and improves application performance
It is achieved by extending the Thread class, implementing the Runnable interface, and using an executor service

Executor Service

ExecutorService is a Java framework used to manage and execute tasks using a thread pool.

Instead of creating threads manually, we submit tasks to ExecutorService, and it manages the available threads for us.
It creates and reuses threads, which improves performance and avoids the overhead of creating a new thread for every task.
execute() → Runs a task but does not return a result.
submit() → Runs a task and returns a Future, which can be used to get the result or check the task status.
We get a Future object and call future.get() to get the result.
If the task fails, we catch an ExecutionException and handle the error properly.
shutdown() → Stops accepting new tasks and allows already submitted tasks to complete.
shutdownNow() → Attempts to stop currently running tasks and returns tasks that were waiting in the queue.
We can create different types of thread pools, such as fixed, cached, or scheduled thread pools.
Suppose we have 100 tasks and a fixed thread pool of 5 threads.
Only 5 tasks run at a time, while the remaining tasks wait in the queue.
When a thread completes one task, it picks up the next task.

synchronization
It is a technique that ensures that only one thread accesses a shared resource at a time, preventing race conditions and maintaining data consistency
types:
synchronized method: locks the entire object, and the entire body is synchronized
synchronized block: locks only a specific section of code
static synchronization(class level lock): locks the class object itself

flatMap
It is used to transform and flatten nested collections or streams into a single stream for easier processing.
Flattening means transforming data from *Stream<Stream<T>> to Stream<T>*.

React

Redux
It is a centralized state management library used to share data across components.
When the user performs an action like clicking the submit button, the component dispatches an action to a reducer.
The reducer updates the state and returns a new state
The updated state is stored in the Store
Components subscribed to the store automatically receive the updated state and are re-rendered.

Component => Dispatch Action => Reducer => Store Updated => UI Re-rendered

Key components:
Store: centralized state container
Action: describes what happened
Reducer: contains logic for the updated state
Dispatch: sends action to reducer

Prop drilling
It means lifting the state of a component up to a higher-level/parent component so that it can be accessed by multiple components.
Disadvantage: Whenever data is needed from the parent component, it comes from each level rather than directly.
It can be avoided using Redux or the React Context API, which allows components to access shared state directly without passing props through intermediate components.

React Context API
Context API is a built-in React feature used to share common data between multiple components.
It helps avoid prop drilling, where we have to pass the same data through multiple component levels.
We create a Context using createContext() and provide the data using a 'Context.Provider'.
Any child component can access that data using useContext() without receiving it through props.
It is simple and works well for small to medium applications where the shared state is not very complex.
For larger applications with complex state management and multiple state updates, Redux can be a better choice.
One important point is that when the Context value changes, the components consuming that context can re-render.

usestate
It is used to manage and update state in functional components
It returns the current state value and a setter function to update the state
Whenever a state is updated, a component is re-rendered
It replaces *this.state* in class components

useeffect
It is used to perform side effects in functional components
It is used to api calls, data fetching, DOM updates, etc
It takes a callback function and a dependency array
[ ] - empty dependency array means it runs only once after initial render
[value] - runs only when value changes
No dependency array means it runs after every render

Virtual DOM

It is an in-memory copy of the actual DOM that React uses to detect changes and efficiently update the modified parts of the UI.
When state or props change, React creates a new virtual DOM.
React compares it with the previous virtual DOM (Diffing).
React identifies only the changed elements and then updates only those changes in the real DOM(Reconciliation).

Managerial

How were feature/business requirement was assigned to you?

Usually, a business requirement is created as a Jira story or task.
The team discusses the requirement, clarifies the details, and estimates the work.
Once the task is assigned to me, I first understand the requirement and acceptance criteria.
Then I work on the development, followed by local testing.
After completing the changes, I push the code to Git and raise a Pull Request for code review.
Once the changes are approved and tested successfully, the ticket is moved to the appropriate completed status.

How was a bug assigned to you?

When a bug is found, a Jira bug ticket is created with the issue details and steps to reproduce it.
Based on the priority and the module involved, the bug is assigned to the appropriate developer.
I first reproduce the issue and check the logs, request data, and related code to find the root cause.
Once I identify the issue, I implement the fix and test it locally.
Then I push the changes to Git and raise a Pull Request for code review.
After the fix is approved and deployed, QA verifies the issue.
If everything works correctly, the Jira bug is closed.